

如何用 Codex 快速入門任何一個領域

打造可迭代的領域學習系統

改寫自 @GeekCatX (知识猫图解)

國立臺中科技大學 王美智

2026 年 6 月

目 錄

- 第 01 章 一句話方法
- 第 02 章 第一步：建立領域學習倉庫
- 第 03 章 第二步：先建立知識地圖，不要直接學細節
- 第 04 章 第三步：讓 Codex 生成「學習 Skill」
- 第 05 章 第四步：每天用「輸入—輸出」方式學習
- 第 06 章 第五步：每個領域都必須做一個小專案
- 第 07 章 第六步：讓 Codex 扮演「考官」，不要只當老師
- 第 08 章 第七步：用「領域壓縮卡」快速複習
- 第 09 章 最強通用提示詞模板
- 第 10 章 推薦的 30 天學習節奏
- 第 11 章 最重要的心法

前言

很多人只把 Codex 當作寫程式碼的工具，但更高效的用法是，把它當成一個可以幫你搭建學習系統、維護知識倉庫、生成練習、檢查成果、推進專案的「領域學習工程師」。

Codex 的優勢不只是回答問題。它可以在專案目錄裡讀寫檔案、執行命令、修改程式碼，同時執行測試、linter、type checker 等自動化檢查。OpenAI 官方文件也明確指出，Codex 能夠在程式碼庫中回答問題、編輯檔案並執行相關命令。更重要的是，Codex 支援透過 AGENTS.md 寫入專案級長期指令——它在開始工作之前會先讀取這個檔案，並且支援全域性指令與專案級指令疊加。此外，Codex Skills 可以將某一類任務封裝成可複用的能力，一個 Skill 通常是一個包含 SKILL.md 的目錄，還可以附帶指令碼和參考資料。

理解了這些能力之後，用 Codex 學一個領域的正確方式就不再是——

請給我講講 AI Agent。

而是讓它幫你建立：

一個可執行、可迭代、可測試、可覆盤的學習倉庫。

第 01 章 一句話方法

蝦哥直接講重點：Codex 不是拿來聊天的，是用來建立專案的。把學習當作一個「可執行的專案」來管理，這才是真正的學習工程師思維。

用 Codex 為某個領域建立一個「學習專案倉庫」，裡面包含知識地圖、核心概念、案例庫、練習題、測驗、專案、覆盤記錄和個人進度。然後讓 Codex 每天根據你的掌握情況，動態調整學習路徑。

這就是 Codex 和普通聊天式學習的分水嶺：

ChatGPT 更適合把知識講明白；Codex 更適合把學習過程工程化、檔案化、專案化、自動化。

► 本章重點

- ► Codex = 領域學習工程師，不是聊天機器人
- ► 核心：建立可運行、可迭代的學習倉庫
- ► 用 AGENTS.md 寫規則給 Codex

? 你目前最想學的一個領域是什麼？試著用一句話描述你想建立的學習倉庫。

第 02 章 第一步：建立領域學習倉庫

跟 Codex 說一句話，它就能幫你生出完整的學習倉庫。最關鍵的檔案叫 *AGENTS.md* ——它是寫給 Codex 看的專案說明書，告訴它怎麼教你。

以學習「AI Agent」為例。在終端中進入工作目錄，啟動 Codex：

然後輸入：

整個倉庫中最關鍵的檔案是 *AGENTS.md*。它不是普通筆記，而是這個學習倉庫的「長期規則檔案」——寫給 Codex 的專案說明書。建議讓 Codex 寫入如下規則：

有了這個檔案，Codex 就不再是一次性問答機器，而是一臺按規則執行的教學引擎。

► 本章重點

- ► 一句話就讓 Codex 建立完整學習倉庫
- ► *AGENTS.md* 是給 Codex 的專案說明書
- ► 包含：知識地圖、核心概念、案例、練習、專案

？如果你要建立一個 AI Agent 學習倉庫，AGENTS.md 裡應該寫哪些規則？

第 03 章 第二步：先建立知識地圖，不要直

接學細節

最怕一上來就扎進資料海。先讓 Codex 幫你畫出知識地圖：20% 必須懂、60% 先跳過、20% 以後補。先有方位感，再學細節。

學習一個新領域，最怕一上來就扎進資料海。正確順序是：

先看地圖 → 再學概念 → 再看案例 → 再做練習 → 最後做專案。

給 Codex 輸入：

你要讓 Codex 做的不是「堆資料」，而是「壓縮領域」。可以讓它把知識拆成三類：

- 必須先懂的 20%：決定你能不能入門。
- 暫時跳過的 60%：現在學了也用不上。
- 以後深入的 20%：做專案後再回來補。

這一步的目標不是學完，而是建立清晰的方位感：我現在在哪裡，我要學什麼，哪些東西先不用管，最後要做出什麼作品。

► 本章重點

- ► 先畫地圖再學細節，不要扎進資料海
- ► 鐵三角：20%必懂 / 60%跳過 / 20%以後補

- ▶ 目標不是學完，是建立方位感

？ 以「深度學習」為例，試著畫出它的 20% 必懂、60% 跳過、20% 以後補。

第 04 章 第三步：讓 Codex 生成「學習

Skill」

學了一個領域，下次學另一個又要重來？讓 Codex 把這套方法封裝成一個 Skill——以後一句話啟動一個完整的領域學習專案。

當你發現自己會反覆學習不同領域時，可以把這套方法封裝成一個可複用的 Codex Skill。輸入：

讓 Codex 生成的 SKILL.md 應該包含清晰的工作流程和每日教學格式。這樣，Codex 就不再只是臨時回答你一次，而是擁有了一套可複用的「學習作業系統」——學一個新領域，一條命令啟動一個完整的學習專案。

► 本章重點

- ► 把學習方法封裝成可複用的 Codex Skill
- ► 一個 SKILL.md 啟動完整學習專案
- ► 相當於擁有自己的「學習操作系統」

？ 你覺得自己的學習過程中，最常卡在哪一步？是知識地圖、練習還是專案？

第 05 章 第四步：每天用「輸入—輸出」方

式學習

不要問「教我機器學習」這種大哉問。要用輸入—輸出的方式：先讓 Codex 給你概念，然後你自己練習、寫答案、複盤。每天用進度檔案追蹤。

不要問「請教我機器學習」——這太寬泛，Codex 很容易給你一篇泛泛而談的說明文。

你應該這樣問：

每天結束時再輸入一段覆盤指令，讓 Codex 總結你掌握了什麼、沒掌握什麼、薄弱點屬於哪一類錯誤（概念不清 / 不會應用 / 表達不清 / 知識混淆），把錯題寫入 progress.md、更新術語表、生成今日知識壓縮卡、安排明天的學習任務。

這樣一來，學習就不再是一次性聊天，而是一個持續演化的專案。Codex 會負責維護 progress.md、術語表、知識卡片、測驗題庫、每日覆盤和專案成果——你只需要每天完成任務、提交答案、覆盤錯誤。

► 本章重點

- ► 用「輸入-輸出」方式取代被動學習
- ► 每天複盤：進度、薄弱點、明日計劃
- ► 用 progress.md + 錯題本 + 知識卡片追蹤

? 設計一個你今天就可以用的「輸入—輸出」學習指令模板。

第 06 章 第五步：每個領域都必須做一個小

專案

真正學會一個領域，不是聽懂了，而是做得出來。讓 Codex 根據你的程度設計最小可行專案——能跑、能展示、能解釋，才算真正學會。

真正掌握一個領域，不是「聽懂了」，而是「能做出來」。用下面的模板讓 Codex 根據你的基礎設計一個最小可行專案：

不同領域可以對應不同的最小專案——AI Agent 可以做一個自動整理資料的 Agent，提示詞工程可以搭建個人提示詞模板庫，資料分析可以做銷售看板，寫作可以做選題-大綱-初稿-修改工作流。專案越小越好，但必須完整：能執行、能展示、能解釋、能迭代、能證明你真的學會了。

► 本章重點

- ► 最小可行專案：能跑、能展示、能解釋
- ► 不同領域對應不同專案類型
- ► 專案越小越好，但必須完整

? 以你熟悉的領域為例，設計一個最小可行專案。

第 07 章 第六步：讓 Codex 扮演「考官」， 不要只當老師

學習最快的人，不是聽最多的，而是被檢測最多的。讓 Codex 扮演考官，出題考你，幫你找到真正的知識缺口。

學習最快的人，不是聽得最多的，而是被檢測最多的。學完一個階段後，輸入：

這一步至關重要。你真正需要的不是更多知識，而是發現：哪些東西你以為懂了其實沒懂？哪些你能聽懂但不能用？哪些你知道概念但說不清楚？哪些是你最該補的短板？

► 本章重點

- ► 讓 Codex 當考官，不只是當老師
- ► 被檢測最多的學最快
- ► 用測驗找出真正的知識缺口

？如果讓 Codex 對你出一份考卷，你最希望它考你什麼？

第 08 章 第七步：用「領域壓縮卡」快速複

習

每天學完不要放著不管。讓 Codex 幫你生成一張「領域壓縮卡」，把當天所學濃縮成一頁精華，30 天後你就有自己的知識資產庫。

每學完一天或一個模組，讓 Codex 生成一張壓縮卡：

30 天后，你會擁有一套獨屬於自己的領域卡片庫。這些卡片不是別人給你的資料，而是根據你的學習過程、錯題和專案經驗生成的個人知識資產。

► 本章重點

- ► 每天一頁「領域壓縮卡」濃縮所學
- ► 30 天後擁有專屬知識卡片庫
- ► 卡片來自你的學習過程，不是複製貼上

? 今天學完一個主題後，試著讓 Codex 幫你生成一張領域壓縮卡。

第 09 章 最強通用提示詞模板

以後學任何領域，直接套用這段通用提示詞模板，Codex 就會自動幫你建立完整的學習專案結構。

以後學習任何領域，都可以直接套用下面這段：

► 本章重點

- ► 通用提示詞模板可複用到任何領域
- ► 一句話啟動完整學習專案
- ► 減少重複勞動，專注在學習本身

? 通用提示詞模板中，你覺得哪個部分對你最有幫助？

第 10 章 推薦的 30 天學習節奏

30 天學習節奏：早上問 Codex 安排任務，晚上做覆盤。每天進步一點，30 天就能完整入門一個新領域。

每天開始時問一句「請根據 progress.md 給我安排今天的學習任務」；每天結束時做一次完整覆盤——總結掌握了什麼、指出沒掌握什麼、判斷薄弱點型別、更新 progress.md 和術語表、生成今日知識卡片、安排明天的任務。

► 本章重點

- ► 早上安排任務，晚上覆盤
- ► 30 天完整入門一個領域
- ► 進度可視化，每一天都看得見成長

? 30 天入門一個領域，你覺得最難堅持的是哪一步？

第 11 章 最重要的心法

用 Codex 學任何領域的核心心法：不是讓它「餵知識」，而是讓它幫你建立五個系統——知識地圖、術語表、練習系統、專案系統、覆盤系統。

用 Codex 學任何領域，不是讓它「喂知識」，而是讓它幫你建立五個系統：

知識地圖——解決「不知道這個領域有什麼」的問題。術語表——解決「聽不懂專業詞」的問題。練習系統——解決「我以為懂了，其實沒懂」的問題。專案系統——解決「學了很多，但不會用」的問題。覆盤系統——解決「學完就忘、錯了不改」的問題。

真正高效的學習方式是：ChatGPT 負責把知識講明白，Codex 負責把學習過程工程化，而你負責每天完成任務、提交答案、覆盤錯誤、推進專案。

最終你得到的，不是一堆碎片筆記，而是一個可以持續迭代的個人學習系統。

► 本章重點

- ► 五大系統：地圖 + 術語 + 練習 + 專案 + 覆盤
- ► ChatGPT 講知識，Codex 工程化
- ► 最終產出：可持續迭代的個人學習系統

？ 五個系統中，你認為哪一個對你當前的學習最有幫助？為什麼？

參考來源

改寫自 @GeekCatX (知识猫图解) 的原文

(<https://x.com/GeekCatX/status/2063523060406423864>)